IteratorManager Class Documentation

Namespace Integration

```
use App\Utilities\Managers\IteratorManger;
```

Methods Overview

The `IteratorManager` class provides various methods in conjunction with the integrated `IteratorTrait` and `RecursiveIteratorTrait`. This allows efficient iteration, traversal, and filtering over both recursive and standard iterators.

Available Methods in IteratorManager

1. setIterator() Method

Sets the current iterator (either standard or recursive) for the IteratorManager.

Method Definition:

```
public function setIterator(?Iterator $iterator):
void
{
    $this->iterator = $iterator; // Direct
assignment, follows best practice
}
```

Explanation:

- **Purpose:** Sets the current iterator to be used by the IteratorManager.
- **Use Case:** Reusing or updating the current iterator for further operations.

Usage Example:

```
$recursiveIterator = $iteratorManager->RecursiveDirectoryIterator('/path/to/directory', [
    'flag' => ['skipDots'],
]);

$iteratorManager->setIterator($recursiveIterator);
```

2. getIterator() Method

Retrieves the current iterator that was previously set using `setIterator()`.

Method Definition:

```
public function getIterator(): ?Iterator
{
    return $this->iterator;
}
```

Explanation:

- **Purpose:** Accesses the current iterator for further processing or iteration.

Usage Example:

```
$iterator = $iteratorManager->getIterator();
foreach ($iterator as $fileInfo) {
    echo $fileInfo->getRealPath() . PHP_EOL;
}
```

3. fetchSettings() Method

Fetches default settings for a specific iterator and applies overrides.

Method Definition:

```
private function fetchSettings(string $iterator,
array $overrides = []): array
{
    $defaultSettings = $this-
>iteratorSettings[$iterator]
      ?? $this->recursiveIteratorSettings[$iterator]
      ?? throw new
IteratorNotFoundException("Settings for iterator
'{$iterator}' not found.");

    return array_replace_recursive($defaultSettings,
array_map(function ($value, $key) use
($defaultSettings) {
      return is_array($value)
        ? array_reduce(
          array_map(fn($subKey) =>
$defaultSettings[$key][$subKey] ?? 0, $value),
          fn($carry, $item) => $carry | $item,
          0
        )
        : $value;
    }, $overrides, array_keys($overrides)));
}
```

Explanation:

- **Purpose:** Ensures the correct flags, modes, and settings are applied for each iterator.

4. resolve() Method

Resolves the class name of the iterator being created.

Method Definition:

```
private function resolve(string $iterator): string
{
    return $this->iteratorSettings[$iterator]
['class']
      ?? $this->recursiveIteratorSettings[$iterator]
['class']
      ?? throw new IteratorNotFoundException("Unknown
iterator: $iterator");
}
```

Explanation:

- **Purpose:** Maps the iterator name to its corresponding class.

5. createIterator() Method

Creates iterators (both recursive and standard) using the resolved class and fetched settings.

Method Definition:

```
private function createIterator(string $iteratorName,
array $settings = [], ...$args): Iterator
{
    try {
        return new ($this->resolve($iteratorName))(
            ...$args,
            ...array_values($this-
>fetchSettings($iteratorName, $settings))
        );
    } catch (Throwable $e) {
        throw new IteratorException("Error creating
{$iteratorName}: " . $e->getMessage(), 0, $e);
    }
}
```

Explanation:

- **Purpose:** Core method for creating iterators.

6. current() Method

Returns the current element in the iterator.

Method Definition:

```
public function current(): mixed
{
    return $this->iterator?->current();
}
```

Explanation:

- **Purpose:** Retrieves the current element from the active iterator.

Usage Example:

```
echo $iteratorManager->current();
```

7. key() Method

Retrieves the key of the current element in the iterator.

Method Definition:

```
public function key(): mixed
{
    return $this->iterator?->key();
}
```

Explanation:

- **Purpose:** Retrieves the key of the current element in the iterator.
- **Return Type:** The return type can vary depending on the iterator (e.g., string for associative arrays, integer for numeric arrays).

Usage Example:

```
$iteratorManager->setIterator($iteratorManager-
```

```
>ArrayIterator(['first' => 1, 'second' => 2], []));

echo "Current key: " . $iteratorManager->key() .
PHP_EOL; // Output: first
$iteratorManager->next();
echo "Next key: " . $iteratorManager->key() .
PHP_EOL; // Output: second
```

8. next() Method

Advances the iterator to the next element.

Method Definition:

```
public function next(): void
{
    $this->iterator?->next();
}
```

Explanation:

- **Purpose:** Advances the iterator to the next element.

9. previous() Method

Moves the iterator to the previous element in bidirectional iterators (if supported).

Method Definition:

```
public function previous(): void
{
    if (method_exists($this->iterator, 'previous')) {
```

```
        $this->iterator->previous();
    } else {
        throw new IteratorException("Previous
operation not supported by this iterator.");
    }
}
```

Explanation:

- **Purpose:** This method moves the iterator to the previous element in bidirectional iterators (e.g., `CachingIterator`), if supported.
- **Error Handling:** Throws an `IteratorException` if the `previous()` method is not supported.

Usage Example:

```
$iteratorManager->setIterator($iteratorManager-
>ArrayIterator([1, 2, 3], []));

// Move forward to the second element
$iteratorManager->next();
echo "Current: " . $iteratorManager->current() .
PHP_EOL; // Output: 2

// Move back to the previous element
$iteratorManager->previous();
echo "Previous: " . $iteratorManager->current() .
PHP_EOL; // Output: 1
```

10. rewind() Method

Rewinds the iterator back to the first element.

Method Definition:

```
public function rewind(): void
{
    $this->iterator?->rewind();
}
```

Explanation:

- **Purpose:** Rewinds the iterator to the first element.

11. valid() Method

Checks if the current position of the iterator is valid.

Method Definition:

```
public function valid(): bool
{
    return $this->iterator?->valid() ?? false;
}
```

Explanation:

- **Purpose:** Verifies if there are still elements to iterate.

12. toArray() Method

Converts any iterator to an array.

Method Definition:

```
public function toArray(?Iterator $iterator, bool
$useKeys = true): array
{
    return iterator_to_array($iterator, $useKeys);
}
```

Explanation:

- **Purpose:** Converts an iterator to an array.

13. count() Method

Counts the number of elements in the iterator.

Method Definition:

```
public function count(?Iterator $iterator): int
{
    return iterator_count($iterator);
}
```

Explanation:

- **Purpose:** Counts the elements in the iterator.

14. applyCallback() Method

Applies a callback function to each element in the iterator.

Method Definition:

```
public function applyCallback(?Iterator $iterator,
callable $callback): int
{
    return iterator_apply($iterator, $callback);
}
```

Explanation:

- **Purpose:** Applies a callback to each element in the iterator.

15. getChildren() Method

Retrieves the children of the current item in a recursive iterator.

Method Definition:

```
public function getChildren(): ?RecursiveIterator
{
    return $this->iterator instanceof
RecursiveIterator
        ? $this->iterator->getChildren()
        : null;
}
```

Explanation:

- **Purpose:** Retrieves the children of the current item in a recursive iterator.

Usage Example:

```
if ($iteratorManager->getChildren()) {
    foreach ($iteratorManager->getChildren() as
$child) {
        echo $child->getRealPath() . PHP_EOL;
    }
}
```

Recursive Iterator-Specific Methods

These methods are useful when working with recursive iterators, such as `RecursiveIteratorIterator` and `RecursiveDirectoryIterator`.

16. getDepth() Method

Retrieves the current depth of a `RecursiveIteratorIterator`.

Method Definition:

```
public function getDepth(): int
{
    return $this->iterator instanceof
RecursiveIteratorIterator
        ? $this->iterator->getDepth()
        : 0;
}
```

Explanation:

- **Purpose:** Retrieves the current depth level of a

`RecursiveIteratorIterator`. Depth starts at `0` for the top level, and increments for nested elements.

- **Default Behavior:** Returns `0` if the current iterator is not a `RecursiveIteratorIterator`.

Usage Example:

```
$directoryIterator = $iteratorManager->RecursiveDirectoryIterator('/path/to/directory', [
    'flag' => ['skipDots'],
]);

$iteratorManager->setIterator($directoryIterator);

foreach ($iteratorManager->getIterator() as $fileInfo) {
    echo "Current depth: " . $iteratorManager->getDepth() . PHP_EOL;
}
```

17. hasChildren() Method

Checks if the current element has children in a `RecursiveIterator`.

Method Definition:

```
public function hasChildren(): bool
{
    return $this->iterator instanceof RecursiveIterator && $this->iterator->hasChildren();
}
```

Explanation:

- **Purpose:** Checks if the current element in a `RecursiveIterator` has child elements (e.g., directories that contain files or subdirectories).
- **Applicability:** Only works with iterators that implement `RecursiveIterator`.

Usage Example:

```
$directoryIterator = $iteratorManager->RecursiveDirectoryIterator('/path/to/directory', [
    'flag' => ['skipDots'],
]);

$iteratorManager->setIterator($directoryIterator);

if ($iteratorManager->hasChildren()) {
    echo "This directory has subdirectories." . PHP_EOL;
}
```

Conclusion

The **IteratorManager** class provides essential functionality for handling both recursive and standard iterators. These methods offer flexible ways to traverse, filter, and manipulate complex directory structures and datasets. With the addition of advanced navigation methods like `previous()` and recursive-specific methods like `getDepth()` and `hasChildren()`, the class allows for greater control over both flat and nested data structures.

Recursive Iterators and Standard Iterators

Recursive Iterators Settings

RecursiveArrayIterator

- **Flags:**
  - `asPropy`
  - `stdProps`

RecursiveCachingIterator

- **Flags:**
  - `fullCache`
  - `callToString`
  - `catchGetChildren`

RecursiveCallbackFilterIterator

- **No flags or modes present**

RecursiveDirectoryIterator

- **Flags:**
  - `skipDots`
  - `unixPaths`
- **Modes:**
  - `asFileInfo`
  - `asPathname`
  - `asSelf`

RecursiveFilterIterator

- **No flags or modes present**

RecursiveIteratorIterator

- **Modes:**
  - `leavesOnly`
  - `selfFirst`
  - `childFirst`

RecursiveRegexIterator

- **Modes:**
  - `match`
  - `replace`
  - `split`
  - `invertMatch`

RecursiveTreeIterator

- **Flags:**
  - `bypassCurrent`
  - `bypassKey`
- **Prefixes:**
  - `left`
  - `midHasNext`
  - `midLast`
  - `endHasNext`
  - `endLast`
  - `right`
- **Modes:**

  - `leavesOnly`
  - `selfFirst`
  - `childFirst`
- **Cache:**
  - `catchGetChild`

Standard Iterators Settings (with Correct Key Names)

ArrayIterator

- **Flags:**
  - `asPropy`
  - `stdProps`

AppendIterator

- **No flags or modes present**

CachingIterator

- **Flags:**

- `fullCache`
- `callToString`
- `catchGetChildren`

DirectoryIterator

- **No flags or modes present**

EmptyIterator

- **No flags or modes present**

FilesystemIterator

- **Modes:**
  - `currentAsFileInfo`
  - `currentAsPathname`
  - `currentAsSelf`
  - `followSymlinks`
  - `keyAsFilename`
  - `keyAsPathname`

- `newCurrentKey`
- `skipDots`
- `unixPaths`

FilterIterator

- **No flags or modes present**

GlobIterator

- **No flags or modes present**

InfiniteIterator

- **No flags or modes present**

IteratorIterator

- **No flags or modes present**

LimitIterator

- **No flags or modes present**

MultipleIterator

- **Flags:**
  - `keysNumeric`
  - `needAll`
  - `needAny`

NoRewindIterator

- **No flags or modes present**

ParentIterator

- **No flags or modes present**

RegexIterator

- **Modes:**
  - `match`
  - `getMatches`

- `allMatches`
- `captureGroups`
- `invertMatch`

SeekableIterator

- **No flags or modes present**

IteratorTrait Methods

The following methods are available in the `IteratorTrait` and provide essential iterator functionality.

1. AppendIterator Method

Method Definition:

```
public function AppendIterator(array $iterators =
[]): AppendIterator
{
    try {
        return array_reduce($iterators, function
($appendIterator, $iterator) {
            $appendIterator->append($iterator);
            return $appendIterator;
        }, $this->createIterator('AppendIterator'));
    } catch (Throwable $e) {
        throw new IteratorException("Error creating
AppendIterator: " . $e->getMessage(), 0, $e);
    }
```

```
}
```

Explanation:

- **Purpose:** This method creates an `AppendIterator` that allows iteration over multiple iterators in sequence, one after another.

Usage Example:

```
$iterator1 = $iteratorManager->ArrayIterator([1, 2,
3], []);
$iterator2 = $iteratorManager->ArrayIterator([4, 5,
6], []);

$appendIterator = $iteratorManager-
>AppendIterator([$iterator1, $iterator2]);

foreach ($appendIterator as $value) {
    echo $value . PHP_EOL;
}
```

2. ArrayIterator Method

Method Definition:

```
public function ArrayIterator(array|object $data =
[], array $settings = []): ArrayIterator
{
    try {
        return $this->createIterator('ArrayIterator',
$settings, $data);
    } catch (Throwable $e) {
        throw new IteratorException("Error creating
ArrayIterator: " . $e->getMessage(), 0, $e);
    }
```

```
}
```

Explanation:

- **Purpose:** This method creates an `ArrayIterator`, which allows iteration over arrays or objects. Flags like `asPropy` allow treating array elements as properties.

Usage Example:

```
$data = ['item1', 'item2', 'item3'];

$settings = ['flag' => ['asPropy']];

$iterator = $iteratorManager->ArrayIterator($data,
$settings);

foreach ($iterator as $key => $value) {
    echo "$key => $value" . PHP_EOL;
}
```

3. CachingIterator Method

Method Definition:

```
public function CachingIterator(Iterator $iterator,
array $settings = []): CachingIterator
{
    try {
        return new ($this-
>resolve('CachingIterator'))(
            $iterator,
            $this->fetchSettings('CachingIterator',
$settings)['flag'] ?? CachingIterator::CALL_TOSTRING
        );
    } catch (Throwable $e) {
```

```
        throw new IteratorException("Error creating
CachingIterator: " . $e->getMessage(), 0, $e);
    }
}
```

Explanation:

- **Purpose:** This method creates a `CachingIterator` that caches the results of the underlying iterator. It supports flags like `fullCache` and `callToString`.

Usage Example:

```
$arrayIterator = $iteratorManager-
>ArrayIterator(['item1', 'item2'], []);

$settings = ['flag' => ['fullCache']];

$cachingIterator = $iteratorManager-
>CachingIterator($arrayIterator, $settings);

foreach ($cachingIterator as $item) {
    echo $item . PHP_EOL;
}
```

4. CallbackFilterIterator Method

Method Definition:

```
public function CallbackFilterIterator(Iterator
$iterator, callable $callback):
CallbackFilterIterator
{
    try {
        return $this-
>createIterator('CallbackFilterIterator', [],
```

```
$iterator, $callback);
    } catch (Throwable $e) {
        throw new IteratorException("Error creating
CallbackFilterIterator: " . $e->getMessage(), 0, $e);
    }
}
```

Explanation:

- **Purpose:** This method creates a `CallbackFilterIterator`, allowing custom filtering during iteration. The filtering logic is provided via a callback function.

Usage Example:

```
$arrayIterator = $iteratorManager->ArrayIterator([1,
2, 3, 4, 5], []);

// Filter callback to only include even numbers
$callback = function ($value) {
    return $value % 2 === 0;
};

// CallbackFilterIterator with the even number filter
$filterIterator = $iteratorManager-
>CallbackFilterIterator($arrayIterator, $callback);

foreach ($filterIterator as $value) {
    echo $value . PHP_EOL;
}
```

5. DirectoryIterator Method

Method Definition:

```
public function DirectoryIterator(string $path = '.',
array $settings = []): DirectoryIterator
{
    try {
        return $this-
>createIterator('DirectoryIterator', $settings,
$path);
    } catch (Throwable $e) {
        throw new IteratorException("Error creating
DirectoryIterator: " . $e->getMessage(), 0, $e);
    }
}
```

Explanation:

- **Purpose:** This method creates a `DirectoryIterator` that allows iterating over the contents of a directory.

Usage Example:

```
$iterator = $iteratorManager->DirectoryIterator('/
path/to/directory', []);

foreach ($iterator as $fileInfo) {
    echo $fileInfo->getFilename() . PHP_EOL;
}
```

6. EmptyIterator Method

Method Definition:

```
public function EmptyIterator(): EmptyIterator
{
    try {
        return $this-
```

```
>createIterator('EmptyIterator');
    } catch (Throwable $e) {
        throw new IteratorException("Error creating
EmptyIterator: " . $e->getMessage(), 0, $e);
    }
}
```

Explanation:

- **Purpose:** This method creates an `EmptyIterator`, which always returns empty results during iteration.

Usage Example:

```
$emptyIterator = $iteratorManager->EmptyIterator();

foreach ($emptyIterator as $value) {
    echo $value . PHP_EOL;  // No output expected
}
```

7. FilesystemIterator Method

Method Definition:

```
public function FilesystemIterator(string $path =
'.', array $settings = []): FilesystemIterator
{
    try {
        return new ($this-
>resolve('FilesystemIterator'))(
            $path,
            $this-
>fetchSettings('FilesystemIterator', $settings)
['mode'] ?? FilesystemIterator::CURRENT_AS_PATHNAME
        );
    } catch (Throwable $e) {
```

```
        throw new IteratorException("Error creating
FilesystemIterator: " . $e->getMessage(), 0, $e);
    }
}
```

Explanation:

- **Purpose:** This method creates a `FilesystemIterator` that provides advanced iteration over files and directories in a directory. It supports modes like `CURRENT_AS_PATHNAME` and `SKIP_DOTS`.

Usage Example:

```
$settings = ['mode' => ['currentAsPathname']];

$filesystemIterator = $iteratorManager-
>FilesystemIterator('/path/to/directory', $settings);

foreach ($filesystemIterator as $filePath) {
    echo $filePath . PHP_EOL;
}
```

8. FilterIterator Method

Method Definition:

```
public function FilterIterator(Iterator $iterator,
callable $callback): FilterIterator
{
    try {
        return new class($iterator, $callback)
extends FilterIterator {
            private $callback;
```

```
            public function __construct($iterator,
$callback)
            {
                parent::__construct($iterator);
                $this->callback = $callback;
            }

            public function accept(): bool
            {
                return ($this->callback)($this-
>current(), $this->key(), $this);
            }
        };
    } catch (Throwable $e) {
        throw new IteratorException("Error creating
FilterIterator: " . $e->getMessage(), 0, $e);
    }
}
```

Explanation:

- **Purpose:** This method creates a `FilterIterator`, which filters items in an iterator based on a custom callback function.

Usage Example:

```
$arrayIterator = $iteratorManager->ArrayIterator([10,
20, 30, 40], []);

// Filter to include only elements greater than 20
$callback = function ($value) {
    return $value > 20;
};

// FilterIterator with custom callback
$filterIterator = $iteratorManager-
```

```
>FilterIterator($arrayIterator, $callback);

foreach ($filterIterator as $value) {
    echo $value . PHP_EOL;
}
```

9. GlobIterator Method

Method Definition:

```
public function GlobIterator(string $pattern = '*',
array $settings = []): GlobIterator
{
    try {
        return $this->createIterator('GlobIterator',
$settings, $pattern);
    } catch (Throwable $e) {
        throw new IteratorException("Error creating
GlobIterator: " . $e->getMessage(), 0, $e);
    }
}
```

Explanation:

- **Purpose:** This method creates a `GlobIterator`, which iterates over files matching a glob pattern (e.g., `*.php` to find all PHP files).

Usage Example:

```
$globIterator = $iteratorManager->GlobIterator('/
path/to/directory/*.php', []);

foreach ($globIterator as $file) {
    echo $file . PHP_EOL;
}
```

10. InfiniteIterator Method

Method Definition:

```
public function InfiniteIterator(Iterator $iterator):
InfiniteIterator
{
    try {
        return $this-
>createIterator('InfiniteIterator', [], $iterator);
    } catch (Throwable $e) {
        throw new IteratorException("Error creating
InfiniteIterator: " . $e->getMessage(), 0, $e);
    }
}
```

Explanation:

- **Purpose:** This method creates an `InfiniteIterator`, which cycles over the underlying iterator infinitely.

Usage Example:

```
$arrayIterator = $iteratorManager->ArrayIterator([1,
2, 3], []);

// InfiniteIterator to repeat the array values
endlessly
$infiniteIterator = $iteratorManager-
>InfiniteIterator($arrayIterator);

foreach (new LimitIterator($infiniteIterator, 0, 10)
as $value) {
    echo $value . PHP_EOL;
}
```

11. IteratorIterator Method

Method Definition:

```
public function IteratorIterator(Traversable
$iterator): IteratorIterator
{
    try {
        return $this-
>createIterator('IteratorIterator', [], $iterator);
    } catch (Throwable $e) {
        throw new IteratorException("Error creating
IteratorIterator: " . $e->getMessage(), 0, $e);
    }
}
```

Explanation:

- **Purpose:** This method creates an `IteratorIterator`, which allows traversal of any `Traversable` object (including iterators or arrays).

Usage Example:

```
$arrayIterator = $iteratorManager-
>ArrayIterator(['one', 'two', 'three'], []);

$iterator = $iteratorManager-
>IteratorIterator($arrayIterator);

foreach ($iterator as $value) {
    echo $value . PHP_EOL;
}
```

12. LimitIterator Method

Method Definition:

```
public function LimitIterator(Iterator $iterator, int
$offset = 0, int $count = -1): LimitIterator
{
    try {
        return $this->createIterator('LimitIterator',
[], $iterator, $offset, $count);
    } catch (Throwable $e) {
        throw new IteratorException("Error creating
LimitIterator: " . $e->getMessage(), 0, $e);
    }
}
```

Explanation:

- **Purpose:** This method creates a `LimitIterator`, which limits the number of items returned from an iterator.

Usage Example:

```
$arrayIterator = $iteratorManager->ArrayIterator([10,
20, 30, 40, 50], []);

// LimitIterator to get only the first 3 elements
$limitIterator = $iteratorManager-
>LimitIterator($arrayIterator, 0, 3);

foreach ($limitIterator as $value) {
    echo $value . PHP_EOL;
}
```

13. MultipleIterator Method

Method Definition:

```
public function MultipleIterator(array $iterators =
[], array $settings = []): MultipleIterator
{
    try {
        return array_reduce($iterators, function
($multipleIterator, $iterator) {
            $multipleIterator-
>attachIterator($iterator);
            return $multipleIterator;
        }, new ($this->resolve('MultipleIterator'))(
            $this->fetchSettings('MultipleIterator',
$settings)['flag'] ?? MultipleIterator::MIT_NEED_ALL
        ));
    } catch (Throwable $e) {
        throw new IteratorException("Error creating
MultipleIterator: " . $e->getMessage(), 0, $e);
    }
}
```

Explanation:

- **Purpose:** This method creates a `MultipleIterator`, which allows iterating over multiple iterators in parallel, yielding tuples (i.e., groups of values from each iterator).

Usage Example:

```
$iterator1 = $iteratorManager-
>ArrayIterator(['apple', 'banana'], []);
$iterator2 = $iteratorManager->ArrayIterator([1, 2,
3], []);

// MultipleIterator to iterate over both iterators in
```

```
parallel
$settings = ['flag' => ['needAll']]; // Only yield if
all iterators have values

$multipleIterator = $iteratorManager-
>MultipleIterator([$iterator1, $iterator2],
$settings);

foreach ($multipleIterator as [$fruit, $number]) {
    echo "$fruit => $number" . PHP_EOL;
}
```

14. NoRewindIterator Method

Method Definition:

```
public function NoRewindIterator(Iterator $iterator):
NoRewindIterator
{
    try {
        return $this-
>createIterator('NoRewindIterator', [], $iterator);
    } catch (Throwable $e) {
        throw new IteratorException("Error creating
NoRewindIterator: " . $e->getMessage(), 0, $e);
    }
}
```

Explanation:

- **Purpose:** This method creates a `NoRewindIterator`, which prevents rewinding of the iterator.

Usage Example:

```
$arrayIterator = $iteratorManager-
```

```
>ArrayIterator([100, 200, 300], []);

// NoRewindIterator to prevent rewinding after
traversal
$noRewindIterator = $iteratorManager-
>NoRewindIterator($arrayIterator);

foreach ($noRewindIterator as $value) {
    echo $value . PHP_EOL;
}

// Attempting to rewind and iterate again won't work
$noRewindIterator->rewind();
foreach ($noRewindIterator as $value) {
    // No output, as rewinding is disabled
}
```

15. ParentIterator Method

Method Definition:

```
public function ParentIterator(RecursiveIterator
$iterator): ParentIterator
{
    try {
        return $this-
>createIterator('ParentIterator', [], $iterator);
    } catch (Throwable $e) {
        throw new IteratorException("Error creating
ParentIterator: " . $e->getMessage(), 0, $e);
    }
}
```

Explanation:

- **Purpose:** This method creates a `ParentIterator`, which is a special type of recursive iterator that only returns

directories that have children.

Usage Example:

```
$recursiveIterator = $iteratorManager-
>RecursiveDirectoryIterator('/path/to/directory',
['flag' => ['skipDots']]);

$parentIterator = $iteratorManager-
>ParentIterator($recursiveIterator);

foreach ($parentIterator as $directory) {
    echo $directory->getRealPath() . PHP_EOL;
}
```

16. RegexIterator Method

Method Definition:

```
public function RegexIterator(Iterator $iterator,
string $regex, array $settings = []): RegexIterator
{
    try {
        return new ($this->resolve('RegexIterator'))(
            $iterator,
            $regex,
            $this->fetchSettings('RegexIterator',
$settings)['mode'] ?? RegexIterator::MATCH
        );
    } catch (Throwable $e) {
        throw new IteratorException("Error creating
RegexIterator: " . $e->getMessage(), 0, $e);
    }
}
```

Explanation:

- **Purpose:** This method creates a `RegexIterator`, which filters items based on a regular expression pattern.

Usage Example:

```
$arrayIterator = $iteratorManager->ArrayIterator(['apple', 'banana', 'cherry'], []);

$regexIterator = $iteratorManager->RegexIterator($arrayIterator, '/a/');

foreach ($regexIterator as $value) {
    echo $value . PHP_EOL;
}
```

17. SeekableIterator Method

Method Definition:

```
public function SeekableIterator(array|object $data = []): SeekableIterator
{
    try {
        return $this->createIterator('SeekableIterator', [], $data);
    } catch (Throwable $e) {
        throw new IteratorException("Error creating SeekableIterator: " . $e->getMessage(), 0, $e);
    }
}
```

Explanation:

- **Purpose:** This method creates a `SeekableIterator`, which allows seeking to a specific position in the iterator.

Usage Example:

```
$data = ['first', 'second', 'third'];

$seekableIterator = $iteratorManager-
>SeekableIterator($data);

$seekableIterator->seek(1);  // Seek to the second
element
echo $seekableIterator->current();  // Output: second
```

RecursiveIteratorTrait Methods

The following methods, available through `RecursiveIteratorTrait`, provide additional functionality for recursive iterators.

1. RecursiveArrayIterator Method

Method Definition:

```
public function RecursiveArrayIterator(array|object
$data = [], array $settings = []):
RecursiveArrayIterator
{
    try {
        return $this-
>createIterator('RecursiveArrayIterator', $settings,
$data);
    } catch (Throwable $e) {
        throw new IteratorException("Error creating
RecursiveArrayIterator: " . $e->getMessage(), 0, $e);
    }
}
```

Explanation:

- **Purpose:** This method creates a `RecursiveArrayIterator` that can recursively iterate over an array or object. Flags like `asPropy` allow treating arrays like objects.

Usage Example:

```
$data = [
    'key1' => ['subkey1' => 'value1', 'subkey2' =>
'value2'],
    'key2' => 'value3',
];

$settings = ['flag' => ['asPropy']];

$iterator = $iteratorManager-
>RecursiveArrayIterator($data, $settings);

foreach (new RecursiveIteratorIterator($iterator,
RecursiveIteratorIterator::SELF_FIRST) as $key =>
$value) {
    echo "$key => $value" . PHP_EOL;
}
```

2. RecursiveCachingIterator Method

Method Definition:

```
public function
RecursiveCachingIterator(RecursiveIterator $iterator,
array $settings = []): RecursiveCachingIterator
{
    try {
```

```
        return $this->createIterator('RecursiveCachingIterator', $settings, $iterator);
    } catch (Throwable $e) {
        throw new IteratorException("Error creating RecursiveCachingIterator: " . $e->getMessage(), 0, $e);
    }
}
```

Explanation:

- **Purpose:** This method creates a `RecursiveCachingIterator`. It caches results from recursive iteration, optimizing performance for repeated access or expensive operations. Flags like `fullCache` allow full caching of all results.

Usage Example:

```
$directoryIterator = $iteratorManager->RecursiveDirectoryIterator('/path/to/directory', ['flag' => ['skipDots']]);

$settings = ['flag' => ['fullCache']];

$cachingIterator = $iteratorManager->RecursiveCachingIterator($directoryIterator, $settings);

foreach ($cachingIterator as $fileInfo) {
    echo $fileInfo . PHP_EOL;
}
```

3. RecursiveCallbackFilterIterator Method

Method Definition:

```
public function
RecursiveCallbackFilterIterator(RecursiveIterator
$iterator, callable $callback):
RecursiveCallbackFilterIterator
{
    try {
        return $this-
>createIterator('RecursiveCallbackFilterIterator',
[], $iterator, $callback);
    } catch (Throwable $e) {
        throw new IteratorException("Error creating
RecursiveCallbackFilterIterator: " . $e-
>getMessage(), 0, $e);
    }
}
```

Explanation:

- **Purpose:** This method creates a `RecursiveCallbackFilterIterator`, which allows filtering items based on a custom callback function. The callback receives each item and determines whether to include it in the iteration.

Usage Example:

```
$directoryIterator = $iteratorManager-
>RecursiveDirectoryIterator('/path/to/directory',
['flag' => ['skipDots']]);

// Custom filter to include only directories
$callback = function ($fileInfo) {
```

```
    return $fileInfo->isDir();
};

$filteredIterator = $iteratorManager-
>RecursiveCallbackFilterIterator($directoryIterator,
$callback);

foreach ($filteredIterator as $fileInfo) {
    echo $fileInfo->getRealPath() . PHP_EOL;
}
```

4. RecursiveDirectoryIterator Method

Method Definition:

```
public function RecursiveDirectoryIterator(string
$path = '.', array $settings = []):
RecursiveDirectoryIterator
{
    try {
        return $this-
>createIterator('RecursiveDirectoryIterator',
$settings, $path);
    } catch (Throwable $e) {
        throw new IteratorException("Error creating
RecursiveDirectoryIterator: " . $e->getMessage(), 0,
$e);
    }
}
```

Explanation:

- **Purpose:** This method creates a `RecursiveDirectoryIterator`, which allows recursive traversal of a directory. It can be configured with flags like `skipDots` to skip special directories and `unixPaths` to treat paths in Unix format.

Usage Example:

```
$settings = ['flag' => ['skipDots', 'unixPaths']];

$iterator = $iteratorManager-
>RecursiveDirectoryIterator('/path/to/directory',
$settings);

foreach ($iterator as $fileInfo) {
    echo $fileInfo->getRealPath() . PHP_EOL;
}
```

5. RecursiveFilterIterator Method

Method Definition:

```
public function
RecursiveFilterIterator(RecursiveIterator $iterator,
callable $callback): RecursiveFilterIterator
{
    try {
        return new class($iterator, $callback)
extends RecursiveFilterIterator {
            private $callback;

            public function __construct($iterator,
$callback)
            {
                parent::__construct($iterator);
                $this->callback = $callback;
            }

            public function accept(): bool
            {
                return ($this->callback)($this-
>current(), $this->key(), $this);
            }
```

```
        };
    } catch (Throwable $e) {
        throw new IteratorException("Error creating
RecursiveFilterIterator: " . $e->getMessage(), 0,
$e);
    }
}
```

Explanation:

- **Purpose:** This method provides a `RecursiveFilterIterator` that applies a custom filtering logic defined in a callback. The callback determines whether to include each item in the iteration.

Usage Example:

```
$directoryIterator = $iteratorManager-
>RecursiveDirectoryIterator('/path/to/directory',
['flag' => ['skipDots']]);

// Filter to only include files modified after a
certain date
$callback = function ($fileInfo) {
    return $fileInfo->getMTime() >
strtotime('2023-01-01');
};

// Custom RecursiveFilterIterator using the callback
$filteredIterator = $iteratorManager-
>RecursiveFilterIterator($directoryIterator,
$callback);

foreach ($filteredIterator as $fileInfo) {
    echo $fileInfo->getRealPath() . PHP_EOL;
}
```

6. RecursiveIteratorIterator Method

Method Definition:

```
public function
RecursiveIteratorIterator(RecursiveIterator
$iterator, array $settings = []):
RecursiveIteratorIterator
{
    try {
        return $this-
>createIterator('RecursiveIteratorIterator',
$settings, $iterator);
    } catch (Throwable $e) {
        throw new IteratorException("Error creating
RecursiveIteratorIterator: " . $e->getMessage(), 0,
$e);
    }
}
```

Explanation:

- **Purpose:** This method creates a `RecursiveIteratorIterator` that flattens a recursive structure into a single-level iterator. Modes like `selfFirst`, `leavesOnly`, and `childFirst` control the order of traversal.

Usage Example:

```
$directoryIterator = $iteratorManager-
>RecursiveDirectoryIterator('/path/to/directory',
['flag' => ['skipDots']]);

$settings = ['mode' => ['selfFirst']];
```

```
$recursiveIterator = $iteratorManager-
>RecursiveIteratorIterator($directoryIterator,
$settings);

foreach ($recursiveIterator as $fileInfo) {
    echo $fileInfo->getRealPath() . PHP_EOL;
}
```

7. RecursiveRegexIterator Method

Method Definition:

```
public function
RecursiveRegexIterator(RecursiveIterator $iterator,
string $regex, array $settings = []):
RecursiveRegexIterator
{
    try {
        return new ($this-
>resolve('RecursiveRegexIterator'))(
            $iterator,
            $regex,
            $this-
>fetchSettings('RecursiveRegexIterator', $settings)
['mode'] ?? RecursiveRegexIterator::MATCH
        );
    } catch (Throwable $e) {
        throw new IteratorException("Error creating
RecursiveRegexIterator: " . $e->getMessage(), 0, $e);
    }
}
```

Explanation:

- **Purpose:** This method creates a `RecursiveRegexIterator`, which allows filtering items

based on a regular expression pattern.

Usage Example:

```
$directoryIterator = $iteratorManager-
>RecursiveDirectoryIterator('/path/to/directory',
['flag' => ['skipDots']]);

$regexIterator = $iteratorManager-
>RecursiveRegexIterator($directoryIterator, '/
\.php$/');

foreach ($regexIterator as $fileInfo) {
    echo $fileInfo->getRealPath() . PHP_EOL;
}
```

8. RecursiveTreeIterator Method

Method Definition:

```
public function
RecursiveTreeIterator(RecursiveIterator $iterator,
array $settings = []): RecursiveTreeIterator
{
    try {
        return $this-
>createIterator('RecursiveTreeIterator', $settings,
$iterator);
    } catch (Throwable $e) {
        throw new IteratorException("Error creating
RecursiveTreeIterator: " . $e->getMessage(), 0, $e);
    }
}
```

Explanation:

- **Purpose:** This method creates a `RecursiveTreeIterator`, which presents items in a tree-like structure. Flags like `bypassCurrent` and modes like `leavesOnly` or `selfFirst` help define the traversal order.

Usage Example:

```
$directoryIterator = $iteratorManager->RecursiveDirectoryIterator('/path/to/directory', ['flag' => ['skipDots']]);

$settings = ['prefix' => ['left', 'midHasNext', 'midLast', 'right'], 'mode' => ['selfFirst']];

$treeIterator = $iteratorManager->RecursiveTreeIterator($directoryIterator, $settings);

foreach ($treeIterator as $fileInfo) {
    echo $fileInfo . PHP_EOL;
}
```

Conclusion

By focusing on the methods available in `IteratorManager` (such as `current()`, `next()`, `rewind()`, `valid()`, etc.), and using them in combination with recursive and standard iterators, you can build powerful traversal, filtering, and iteration logic.

SPL FileInfo-Related Methods in

IteratorManager

The following methods integrate SPL FileInfo functionalities with iterators, allowing you to manipulate and retrieve detailed information about files and directories.

FileInfo() Method

Creates and returns a `SplFileInfo` object for the given file path, allowing you to access detailed information about the file.

Method Definition:

```
public function FileInfo(string $filePath):
SplFileInfo
{
    return new SplFileInfo($filePath);
}
```

Explanation:

- **Purpose:** This method creates and returns an `SplFileInfo` object for the given file path. This allows you to access detailed information about a file, such as its size, permissions, and real path.
- **Use Case:** Useful for performing file system operations on individual files by providing detailed metadata.

Usage Example:

```
$fileInfo = $iteratorManager->FileInfo('/path/to/
file.txt');

echo "File Size: " . $fileInfo->getSize() . "
bytes" . PHP_EOL;
echo "File Permissions: " . decoct($fileInfo-
>getPerms() & 0777) . PHP_EOL;
```

1. getRealPath() Method

Retrieves the absolute path of the current file or directory.

Purpose:

- Retrieves the absolute path of the current file or directory.

Usage Example with RecursiveDirectoryIterator:

```
$recursiveIterator = $iteratorManager-
>RecursiveDirectoryIterator('/path/to/directory', [
    'flag' => ['skipDots'],
]);

foreach ($recursiveIterator as $fileInfo) {
    echo "File Real Path: " . $fileInfo-
>getRealPath() . PHP_EOL;
}
```

Usage Example with DirectoryIterator:

```
$directoryIterator = $iteratorManager-
>DirectoryIterator('/path/to/directory', []);
```

```
foreach ($directoryIterator as $fileInfo) {
    echo "Real Path: " . $fileInfo->getRealPath() .
PHP_EOL;
}
```

2. getSize() Method

Returns the size of the current file in bytes.

Purpose:

- Returns the size of the current file in bytes.

Usage Example with RecursiveDirectoryIterator:

```
$recursiveIterator = $iteratorManager-
>RecursiveDirectoryIterator('/path/to/directory', [
    'flag' => ['skipDots'],
]);

foreach ($recursiveIterator as $fileInfo) {
    if ($fileInfo->isFile()) {
        echo "File: " . $fileInfo->getFilename() . "
Size: " . $fileInfo->getSize() . " bytes" . PHP_EOL;
    }
}
```

Usage Example with SPL FileInfo and CallbackFilterIterator:

```
$directoryIterator = $iteratorManager-
>DirectoryIterator('/path/to/directory', []);

$callback = function (SplFileInfo $fileInfo) {
    return $fileInfo->getSize() > 500;
};
```

```
$filterIterator = $iteratorManager-
>CallbackFilterIterator($directoryIterator,
$callback);

$iteratorManager->applyCallback($filterIterator,
function ($iterator) {
    echo "File: " . $iterator->current()-
>getFilename() . " is larger than 500 bytes." .
PHP_EOL;
    return true;
});
```

3. getPerms() Method

Returns the permissions of the current file in octal format.

Purpose:

- Returns the permissions of the current file in octal format.

Usage Example with RecursiveDirectoryIterator:

```
$recursiveIterator = $iteratorManager-
>RecursiveDirectoryIterator('/path/to/directory', [
    'flag' => ['skipDots'],
]);

foreach ($recursiveIterator as $fileInfo) {
    echo "File: " . $fileInfo->getFilename() . "
Permissions: " . decoct($fileInfo->getPerms() &
0777) . PHP_EOL;
}
```

Usage Example with CallbackFilterIterator:

```
$arrayIterator = $iteratorManager->ArrayIterator(['/
```

```
path/to/file1', '/path/to/file2'], []);

$callback = function ($filePath) use
($iteratorManager) {
    $fileInfo = $iteratorManager-
>FileInfo($filePath);
    return $fileInfo->getPerms() === 0755;
};

$filterIterator = $iteratorManager-
>CallbackFilterIterator($arrayIterator, $callback);

foreach ($filterIterator as $filePath) {
    echo "File: " . $filePath . " has 0755
permissions." . PHP_EOL;
}
```

4. isFile() Method

Determines whether the current item is a file.

Purpose:

- Determines whether the current item is a file.

Usage Example with RecursiveDirectoryIterator:

```
$recursiveIterator = $iteratorManager-
>RecursiveDirectoryIterator('/path/to/directory', [
    'flag' => ['skipDots'],
]);

foreach ($recursiveIterator as $fileInfo) {
    if ($fileInfo->isFile()) {
        echo "File: " . $fileInfo->getFilename() .
PHP_EOL;
```

```
    }
}
```

Usage Example with MultipleIterator:

```
$iterator1 = $iteratorManager->ArrayIterator(['/path/
to/file1', '/path/to/file2'], []);
$iterator2 = $iteratorManager->ArrayIterator(['/path/
to/file3', '/path/to/file4'], []);

$multipleIterator = $iteratorManager-
>MultipleIterator([$iterator1, $iterator2], []);

foreach ($multipleIterator as $paths) {
    foreach ($paths as $path) {
        $fileInfo = $iteratorManager-
>FileInfo($path);
        if ($fileInfo->isFile()) {
            echo "File: " . $fileInfo-
>getFilename() . PHP_EOL;
        }
    }
}
```

5. isDir() Method

Checks if the current item is a directory.

Purpose:

- Checks if the current item is a directory.

Usage Example with RecursiveTreeIterator:

```
$directoryIterator = $iteratorManager-
>RecursiveDirectoryIterator('/path/to/directory', [
```

```
    'flag' => ['skipDots'],
]);

$treeIterator = $iteratorManager-
>RecursiveTreeIterator($directoryIterator, [
    'prefix' => ['left', 'midHasNext', 'midLast',
'right'],
]);

foreach ($treeIterator as $fileInfo) {
    if ($fileInfo->isDir()) {
        echo "Directory: " . $fileInfo-
>getFilename() . PHP_EOL;
    }
}
```

Combining Utility Methods with SPL FileInfo

These utility methods leverage SPL FileInfo objects to enhance file and directory management.

1. toArray() + SPL FileInfo

Converts the iterator to an array and combines it with SPL FileInfo to get detailed file properties.

Purpose:

- Converts the iterator to an array and combines it with SPL FileInfo to retrieve detailed file properties.

Usage Example with RecursiveDirectoryIterator:

```
$directoryIterator = $iteratorManager->RecursiveDirectoryIterator('/path/to/directory', [
    'flag' => ['skipDots'],
]);

$filesArray = $iteratorManager->toArray($directoryIterator);

$filesWithDetails = array_map(function ($fileInfo) {
    return [
        'filename' => $fileInfo->getFilename(),
        'size' => $fileInfo->getSize(),
        'realPath' => $fileInfo->getRealPath(),
    ];
}, $filesArray);

print_r($filesWithDetails);
```

2. applyCallback() + SPL FileInfo

Applies a callback function to each element in the iterator and uses SPL FileInfo to filter files based on their properties.

Purpose:

- Applies a callback function to each element in the iterator, and uses SPL FileInfo to filter files based on their properties.

Usage Example with RecursiveCallbackFilterIterator:

```
$directoryIterator = $iteratorManager->RecursiveDirectoryIterator('/path/to/directory', [
    'flag' => ['skipDots'],
]);
```

```
$callback = function ($fileInfo) {
    return $fileInfo->getSize() > 1000;
};

$filterIterator = $iteratorManager-
>RecursiveCallbackFilterIterator($directoryIterator,
$callback);

$iteratorManager->applyCallback($filterIterator,
function ($iterator) {
    echo "Large file: " . $iterator->current()-
>getFilename() . " Size: " . $iterator->current()-
>getSize() . " bytes" . PHP_EOL;
    return true;
});
```

3. count() + SPL FileInfo

Counts the number of elements in the iterator and filters files based on specific criteria (e.g., only files, only directories).

Purpose:

- Counts the number of elements in the iterator and filters files based on specific criteria (e.g., only files, only directories).

Usage Example with RecursiveDirectoryIterator:

```
$directoryIterator = $iteratorManager-
>RecursiveDirectoryIterator('/path/to/directory', [
    'flag' => ['skipDots'],
]);

$filesArray =
```

```php
array_filter(iterator_to_array($directoryIterator),
function ($fileInfo) {
    return $fileInfo->isFile();
});

$fileCount = count($filesArray);

echo "Number of files: $fileCount" . PHP_EOL;
```

Conclusion

By integrating SPL FileInfo-related methods (`getRealPath()`, `getSize()`, `getPerms()`, `isFile()`, `isDir()`) with iterator methods like `toArray()`, `applyCallback()`, and `count()`, **IteratorManager** provides powerful functionality for managing and manipulating file system contents.